

LIF — Leaky Integrate-and-Fire Neuron

Module Documentation — Shaaban Unit / MultiStage-DeepSNN

Module name	LIF (Leaky Integrate-and-Fire neuron)
Purpose	Final stage of each Shaaban unit — converts a continuous-valued pooled input into a binary spike decision every clock cycle.
Written by	Unknown (original author not recorded)
Revised by	Mohamed Hussein
Date	July 6, 2026

1. Purpose

The LIF module is the final stage of each Shaaban unit, following the Bias+ReLU → BatchNorm → MaxPool chain. It receives one pooled, continuous-valued sample per cycle and decides whether that channel's neuron fires. One LIF instance is used per Shaaban-unit output, and its 1-bit spike output is what actually propagates forward through the network — everything upstream of it (convolution, bias, batchnorm, pooling) only exists to produce the value this neuron integrates.

Functionally, it implements a discrete-time Leaky Integrate-and-Fire (LIF) neuron: a running membrane potential that leaks over time, accumulates new input every cycle, and resets itself for one cycle after firing.

2. Biological & Circuit Analogy

The LIF neuron is conventionally modelled as a simple RC (resistor–capacitor) circuit driven by an injected current, which maps directly onto the hardware's signal names:

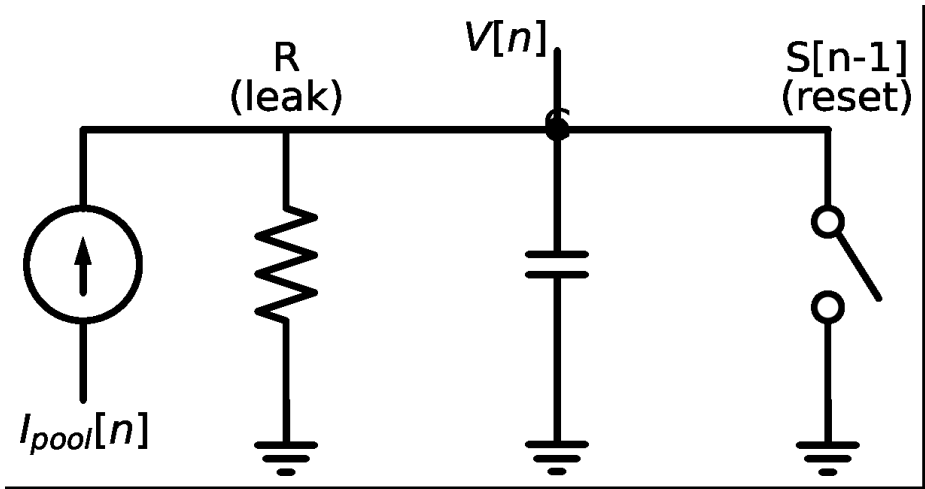


Figure 1. RC circuit analogy. $I_{pool}[n]$ is the injected current (the pooled input); R models the passive leak of charge each cycle; C models the membrane's charge storage (the potential $V[n]$); the switch models the threshold-triggered discharge (reset) applied one cycle after a spike.

Circuit element	Hardware signal / operation	Role
Injected current	in_pool (I_pool[n])	New charge added to the membrane every cycle
Resistor R	mem_reg >>> 1 (mem_leak)	Passive leak — halves the stored potential every cycle
Capacitor C	mem_reg (V[n])	Stores/integrates the membrane potential across cycles
Discharge switch	reset_val, gated by spike_reg	Drains the membrane by exactly one threshold, one cycle after firing

3. Behavioural Model

At the network level, a neuron j sums weighted spikes arriving from upstream neurons i through synapses w_{ij} . Each arriving contribution nudges the membrane potential upward; between arrivals the potential leaks back down. Once the membrane crosses the firing threshold v_{th} , the neuron emits an output spike and its potential is reset.

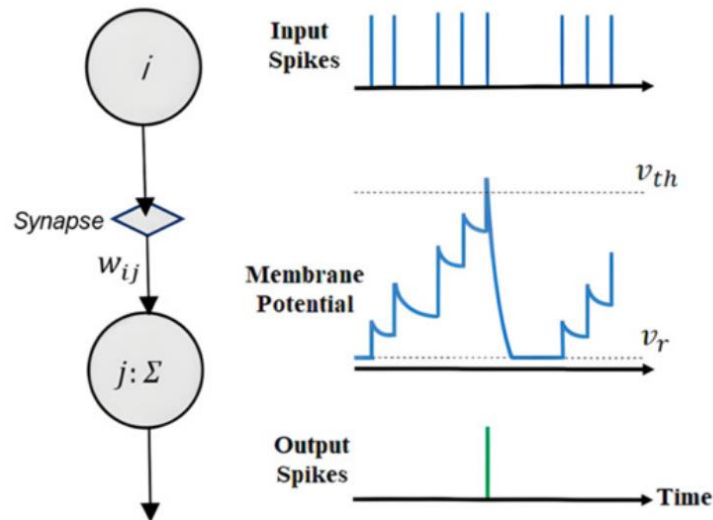


Figure 2. Generic LIF behaviour. Input spikes (top) accumulate into a rising, leaking membrane potential (middle); once v_{th} is crossed the neuron fires a single output spike (bottom) and the potential resets.

In this hardware module, the same shape of behaviour applies, with two implementation-specific details: (1) the “input spikes” in Figure 2 correspond here to `in_pool`, a continuous-valued sample rather than a binary spike, and (2) the reset is not a soft return to a fixed v_r — it is a hard subtraction of the full threshold value, applied one cycle after the spike that triggered it (see Section 4).

4. Core Mathematical Model (per cycle)

The neuron implements a discrete-time LIF model with 50% decay and a one-cycle-delayed hard reset. Using V for membrane potential, I for input, and S for spike, the per-cycle update is:

```

V_leak[n] = V[n-1] >>> 1                (x0.5 decay)    ... (1)
V_sum[n]  = V_leak[n] + I_pool[n]         ... (2)
Reset[n]  = { threshold , if S[n-1] = 1      ... (3)
              { 0 ,           otherwise
V[n]      = V_sum[n] - Reset[n]           ... (4)
S[n]      = { 1 , if V[n] >= threshold      ... (5)
              { 0 , otherwise

```

Equation (1) is the leak — implemented as an arithmetic right-shift (`mem_leak`), which preserves sign while halving magnitude. Equation (2) integrates the new input on top of the leaked potential (`mem_input_add`, truncated back to 18 bits as `mem_input_add_trnuc`). Equation (3) is the delayed reset selection: it looks at `spike_reg`, i.e. whether the neuron fired last cycle, not this one. Equation (4) applies that reset to produce the next membrane potential (`new_mem`). Equation (5) is the threshold comparison that produces the spike decision (`spike / spike_int`), which is registered on the next clock edge into both `mem_reg` and `spike_reg`.

Implementation note — delayed reset: because `Reset[n]` depends on `S[n-1]` rather than `S[n]`, the membrane potential is allowed to keep accumulating for one extra cycle after crossing the threshold before the reset is actually subtracted. This is a one-cycle overshoot inherent to the current design, not a bug; it should be accounted for in any testbench that checks post-spike membrane values rather than just spike timing.

5. Port Description

Signal	Dir.	Width	Description
<code>clk</code>	input	1	System clock.
<code>arst_n</code>	input	1	Asynchronous, active-low reset. Clears the membrane potential (<code>mem_reg</code>) and spike history (<code>spike_reg</code>) immediately when low, independent of <code>clk</code> . Confirmed and implemented — matches the reset convention used elsewhere in the design.
<code>in_pool</code>	input	18 (signed)	Pooled Bias+ReLU+BatchNorm sample for this cycle — <code>I_pool[n]</code> in Section 4.
<code>spike</code>	output	1	Registered spike decision for this cycle — <code>S[n]</code> in Section 4.

Internal signals

Signal	Corresponds to	Description
mem_reg	V[n-1]	Registered membrane potential from the previous cycle.
spike_reg	S[n-1]	Registered spike flag from the previous cycle; gates the delayed reset.
mem_leak	V_leak[n]	Membrane potential after the 50% leak (Eq. 1).
mem_input_add / _trnuc	V_sum[n]	Leaked potential plus new input, before/after truncation to 18 bits (Eq. 2).
reset_val	Reset[n]	Amount subtracted this cycle — threshold if the neuron fired last cycle, else 0 (Eq. 3). Renamed from mux_out (see Section 6).
new_mem	V[n]	Next membrane potential, registered into mem_reg on the next clock edge (Eq. 4).
spike_int	S[n]	Combinational spike decision, registered into spike_reg on the next clock edge (Eq. 5).

6. Implementation & Revision Notes

Changes made during this readability/testbench-prep revision pass:

- Converted always → always_ff / always_comb; file renamed LIF.v → LIF.sv (SystemVerilog syntax required for these keywords).
- All port and internal signal types converted from wire/reg to logic.
- Renamed mux_out → reset_val: the original name was misleading, since it is the internal delayed-reset subtractor (Reset[n]), not the module's actual output (spike).
- Reset polarity/timing: changed from synchronous, active-high (rst) to asynchronous, active-low (arst_n) — confirmed, and now matches the reset convention used elsewhere in the project.

7. Revision Log

Date	Author	Change
July 5, 2026	Mohamed Hussein	Added module header block; converted always blocks to always_ff/always_comb; converted wire/reg to logic; added staged ('===') comments before each critical stage.
July 6, 2026	Mohamed Hussein	Renamed mux_out → reset_val for clarity; converted reset to asynchronous active-low (arst_n) in the RTL; authored this documentation.
July 6, 2026	Mohamed Hussein	Reset change confirmed — documentation updated to reflect arst_n (asynchronous, active-low) as final.